

Predicting Smartphone Addiction with Logistic Regression

Carlos Manuel Martínez Vázquez
Instituto Tecnológico y de Estudios Superiores de Monterrey

August 30, 2026

Abstract

This work presents the implementation of a logistic regression model for a binary classification problem, whose purpose is to predict smartphone addiction from usage habits. The dataset comes from Kaggle’s *Playground Series S6E8* competition and contains 691,369 training records with 4% to 19% missing values across every column. This first iteration covers the data preparation and the initial model. The model achieved an AUC of 0.911 on the training set.

1 Introduction

Excessive smartphone use has become a public health concern associated with sleep disorders, anxiety and poor academic performance. Identifying people with problematic usage patterns early would allow preventive interventions to be designed.

This work approaches the problem as a binary classification task: given nine numeric variables describing usage habits and three categorical context variables, the goal is to predict the probability that a person exhibits smartphone addiction.

Since the dataset contains missing values in every one of its columns, this first deliverable centres on the data preparation stage, paying particular attention to statistically justifying the imputation strategy rather than assuming a default criterion. The model itself was implemented directly in *NumPy*, which keeps every step of the algorithm explicit and allows the gradient to be verified numerically.

2 ETL

2.1 Data source

The data comes from the *Predicting Smartphone Addiction* competition (Playground Series, Season 6, Episode 8) on Kaggle. The dataset is distributed already split into two files:

- `train.csv`: 691,369 records and 14 columns.

- `test.csv`: 296,302 records and 13 columns.

The column missing from `test.csv` is the target variable `addicted_label`, since the test labels are withheld by the platform in order to score submitted predictions. For this reason, 100% of `train.csv` was used for training.

The target variable is binary and moderately imbalanced: 490,474 positive cases (70.94%) against 200,895 negative ones (29.06%). No duplicate records were found.

2.2 Variable description

Table 1 describes the available variables.

Variable	Type
<code>age</code>	Numeric
<code>daily_screen_time_hours</code>	Numeric
<code>social_media_hours</code>	Numeric
<code>gaming_hours</code>	Numeric
<code>work_study_hours</code>	Numeric
<code>sleep_hours</code>	Numeric
<code>notifications_per_day</code>	Numeric
<code>app_opens_per_day</code>	Numeric
<code>weekend_screen_time</code>	Numeric
<code>gender</code>	Categorical
<code>stress_level</code>	Categorical
<code>academic_work_impact</code>	Categorical
<code>addicted_label</code>	Target (binary)

Table 1: Dataset variables.

2.3 Exploratory analysis

The correlation matrix (Figure 1) reveals two relevant findings.

First, the variables most correlated with the target are `daily_screen_time_hours` (0.61), `weekend_screen_time` (0.59) and `social_media_hours` (0.53). After these three there is a sharp drop: `work_study_hours` (0.25) and `gaming_hours` (0.21), while `age`, `sleep_hours`,

notifications_per_day and app_opens_per_day show correlations of 0.06 or lower.

Second, those same three screen-time variables are strongly correlated *with each other*, the highest being between daily_screen_time_hours and weekend_screen_time (0.80).

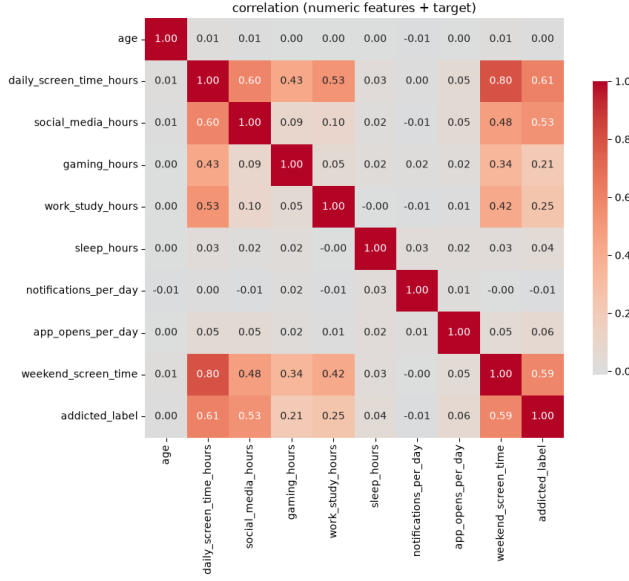


Figure 1: Correlation matrix of the numeric variables and the target.

2.4 Missing values

Every column in the training set contains missing values, in proportions ranging from 4.18% to 19.38% (Table 2). This is the main data quality problem of the project.

Variable	% missing
social_media_hours	19.38
gaming_hours	18.34
weekend_screen_time	16.21
daily_screen_time_hours	13.86
app_opens_per_day	11.67
notifications_per_day	9.78
stress_level	7.98
work_study_hours	7.45
sleep_hours	6.43
academic_work_impact	6.40
gender	4.20
age	4.18

Table 2: Percentage of missing values per variable.

Simple imputation was chosen to fill these values. The choice of the statistic used for the imputation is

based on the skewness test presented in the following section.

2.5 Skewness test and choice of the median

The choice between mean and median as the imputation statistic depends on the symmetry of the distribution. The mean is sensitive to outliers, since it is dragged toward the long tail of a skewed distribution, whereas the median is robust to them. The skewness coefficient of each numeric variable was therefore computed (Table 3), using the conventional threshold of $|g| > 1$ to consider a distribution strongly skewed.

Variable	Skewness
work_study_hours	0.56
gaming_hours	0.54
social_media_hours	0.47
notifications_per_day	-0.21
daily_screen_time_hours	-0.13
app_opens_per_day	-0.13
weekend_screen_time	-0.06
age	-0.04
sleep_hours	-0.01

Table 3: Skewness coefficient per numeric variable.

No variable exceeds an absolute skewness of 0.56, so all distributions can be considered approximately symmetric. Figure 2 illustrates the effect of skewness on the separation between mean and median: even at a skewness of 1.0 (higher than that of any variable in this dataset) the difference between the two statistics is only 0.15 standard deviations, and it vanishes in the symmetric case.

The **median** was selected for three reasons:

1. In symmetric distributions the median coincides with the mean, so using it costs nothing; in exchange, it offers protection should the symmetry not hold.
2. The imputed value is replicated across up to 133,995 records (social_media_hours). The median is a value actually observed in the data, whereas the mean may not be: in notifications_per_day, an integer-valued variable, the mean is 145.89 notifications, a quantity no real record can hold.
3. It matches the behaviour of standard imputation utilities, which keeps later iterations of this work directly comparable.

For the three categorical variables the mode was used, since the median is meaningless for these types of vari-

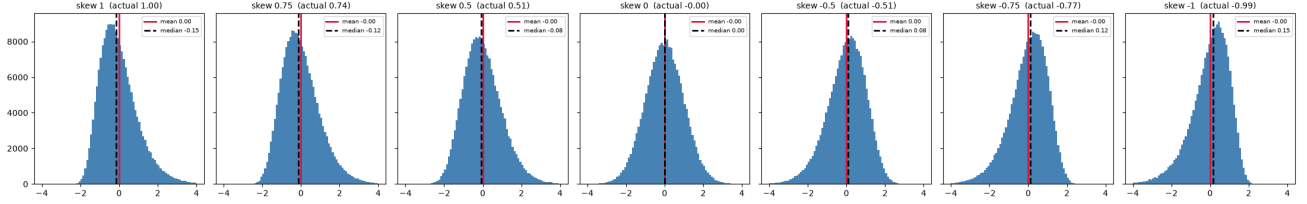


Figure 2: Simulated distributions with controlled skewness, standardized to mean zero and standard deviation one. The red line marks the mean and the dashed black line the median. As skewness increases, the mean shifts toward the long tail while the median stays in place.

ables, as they classify rather than being numerical values.

2.6 Scaling and clipping of extreme values

The numeric variables were standardized using the z -score:

$$z = \frac{x - \mu}{\sigma} \quad (1)$$

This step is essential because the variables are on heterogeneous scales: `notifications_per_day` has a standard deviation of 65.92 against 0.93 for `gaming_hours`, a difference of roughly 70 times. Without standardization, principal component analysis would be dominated by the higher-magnitude variables purely because of their units of measurement.

Values were then clipped at ± 3 standard deviations. This criterion was chosen over the interquartile range (IQR) based on the result of the skewness test: the $\pm 3\sigma$ rule assumes an approximately symmetric distribution, a condition satisfied by this dataset, where the maximum skewness is 0.56. The IQR criterion ($Q_1 - 1.5 \text{ IQR}$, $Q_3 + 1.5 \text{ IQR}$) is preferable for strongly skewed distributions, since it makes no symmetry assumption; with no relevant skewness present, it would offer no advantage over the simpler criterion.

Clipping modified only 0.066% of the values, roughly one in every 1,500. This confirms that the distributions were already naturally bounded, so the operation acts as a low-cost preventive measure rather than a substantive correction.

2.7 Encoding of categorical variables

The three categorical variables were one-hot encoded, dropping the first category of each.

2.8 Principal component analysis

PCA was implemented manually through the eigendecomposition of the covariance matrix of the standardized data, using `numpy.linalg.eigh`, which is appro-

priate for symmetric matrices. The components were sorted from highest to lowest explained variance.

	Explained variance	Cumulative variance
PC1	0.2747	0.2747
PC2	0.1160	0.3907
PC3	0.1114	0.5021
PC4	0.1100	0.6122
PC5	0.1080	0.7202
PC6	0.1055	0.8257
PC7	0.0998	0.9255
PC8	0.0485	0.9740
PC9	0.0260	1.0000

Table 4: Variance explained by each principal component and cumulative variance.

Table 4 and Figure 3 show the variance captured by each component. Only PC1 stands out, explaining 27.47% of the variance, roughly two and a half times what a single variable would contribute; its highest loadings correspond to `daily_screen_time_hours` (-0.58), `weekend_screen_time` (-0.54) and `social_media_hours` (-0.39). Squaring each loading gives that variable’s share of the component, so these three account for $0.58^2 + 0.54^2 + 0.39^2 \approx 80\%$ of PC1, with the remainder spread thinly across the other six variables. PC1 therefore corresponds to the correlated block identified in the exploratory analysis, collapsed into a single axis of overall screen usage.

The first seven components were retained, preserving 92.55% of the total variance. The final input vector to the model consists of twelve columns: seven principal components and five binary variables.

3 Logistic Regression

3.1 Sigmoid function

The sigmoid function transforms the linear combination of features and parameters into a probability within the interval $(0, 1)$:

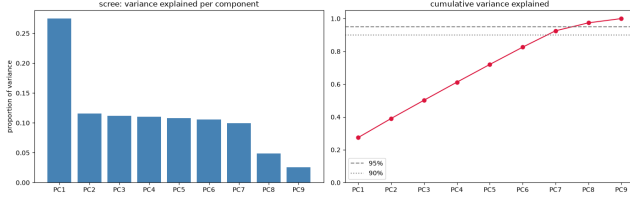


Figure 3: Variance explained per component and cumulative variance.

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}} \quad (2)$$

In the implementation, the argument of the exponential is clipped to the interval $[-500, 500]$ to avoid numerical overflow of the exponential function for very negative values.

3.2 Cost function

Binary cross-entropy was used:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(h_i) + (1 - y_i) \log(1 - h_i)] \quad (3)$$

Probabilities are clipped to the interval $[10^{-10}, 1 - 10^{-10}]$ before evaluating the logarithm, thereby avoiding the $\log(0)$ indeterminacy.

3.3 Gradient descent

The parameters are updated iteratively through:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_i - y_i) x_{ij} \quad (4)$$

where α is the learning rate, m the number of samples, h_i the prediction for sample i and y_i its true label.

A column of ones was appended to the feature matrix, so that θ_0 acts as the intercept term. The parameters were initialized at zero, except for the intercept, which was assigned the log-odds of the base rate:

$$\theta_0 = \log \frac{\bar{y}}{1 - \bar{y}} = 0.8926 \quad (5)$$

In this way the model starts out correctly predicting the average proportion of the positive class, and gradient descent only has to learn the deviations from that baseline.

3.4 Gradient checking

To verify that the analytical expression of the gradient was implemented correctly, it was compared against a numerical approximation by central differences:

$$\frac{\partial J}{\partial \theta_j} \approx \frac{J(\theta_j + \epsilon) - J(\theta_j - \epsilon)}{2\epsilon} \quad (6)$$

With $\epsilon = 10^{-5}$ on the parameter corresponding to PC1, the numerical gradient was 2.1495306×10^{-7} and the analytical one 2.1495081×10^{-7} , agreeing to six significant figures. This test confirms the correctness of the implementation independently of whether the cost decreases during training.

3.5 Training process

The model was trained on the entire training set (691,369 records and 12 features) with a learning rate of $\alpha = 0.5$. The stopping criterion was set to when the parameters stop changing appreciably between consecutive iterations (absolute tolerance of 10^{-6}).

It is worth noting that an exact equality comparison between parameter vectors is never satisfied when working with floating-point numbers, since gradient descent keeps producing changes in the last decimal places indefinitely; a tolerance-based comparison was therefore used.

Training stopped at epoch 1,678. The cost decreased from 0.60267 to 0.34658, as shown in Figure 4. The initial value corresponds exactly to the entropy of a 71/29 distribution, which verifies the correct initialization of the intercept.

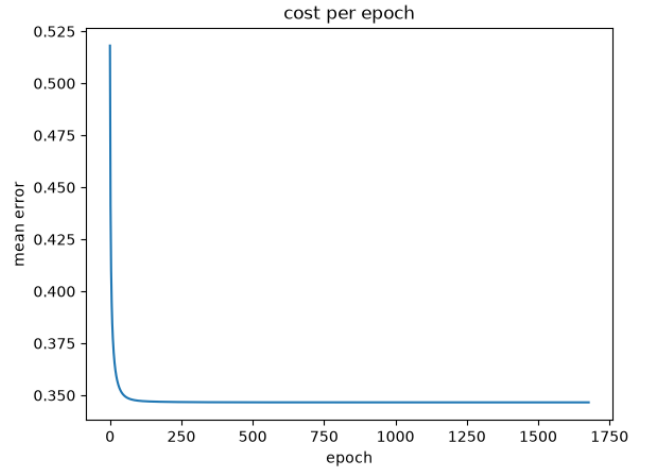


Figure 4: Evolution of the cost during training.

3.6 Results

The evaluation metric of the competition is the area under the ROC curve (AUC), computed on the predicted probabilities. Since AUC depends only on the ordering of the predictions and not on a decision threshold, there was no need to tune such a threshold or to correct the class imbalance.

AUC was implemented manually from its equivalence with the Mann-Whitney U statistic:

$$\text{AUC} = \frac{R_+ - \frac{n_+(n_++1)}{2}}{n_+ \cdot n_-} \quad (7)$$

where R_+ is the sum of the ranks of the positive observations, and n_+ and n_- the number of positive and negative observations respectively.

Metric	Value
AUC (training)	0.911
Final cost $J(\theta)$	0.34658
Baseline cost	0.60267
Epochs to convergence	1,678

Table 5: Results of the logistic regression model.

The model achieved an **AUC of 0.911** (Table 5). As a validity check on the metric implementation, AUC was evaluated on the same predictions in random order, yielding 0.50073, a value consistent with what is expected for a random classifier.

The final cost of 0.34658 represents a reduction of roughly 43% against the cost of 0.60267 corresponding to a model that always predicts the base rate. It is important to note that this value is not an error rate: cross-entropy is a measure of confidence and is not bounded above. Its direct interpretation is that the model assigns on average a probability of $e^{-0.34658} = 0.707$ to the correct class.

The cost cannot approach zero because the distributions of the two classes overlap considerably, as observed in the exploratory analysis: there are records with identical screen usage levels in both classes, so a portion of the error is irreducible.

4 Conclusions

A logistic regression model was successfully implemented and trained, achieving an AUC of 0.911 on the training set.

The treatment of missing values was the central aspect of this work. The skewness test provided a quantitative justification for using the median instead of the mean, as well as for preferring the ± 3 standard deviation criterion over the interquartile range when clipping extreme values. In both cases, the decision rested on a measurement rather than on a convention.

Principal component analysis showed that the dataset admits no relevant dimensionality reduction: seven of nine components were needed to retain 92.55% of the variance. This result is a finding in itself, since it shows that the variables are mostly independent, with the exception of the correlated screen-time block.

Future work includes an equivalent implementation built on an established library, in order to compare both approaches, as well as the addition of a confusion matrix, hyperparameter tuning and cross-validation. A different algorithm could also be applied to this same problem, will try to do it if I have enough time haha.

References

- [1] Kaggle. *Playground Series S6E8: Predicting Smartphone Addiction*. <https://www.kaggle.com/competitions/playground-series-s6e8>
- [2] A. Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd ed., O'Reilly Media, 2022.
- [3] GeeksforGeeks. *ML | Handling Missing Values*. <https://www.geeksforgeeks.org/machine-learning/ml-handling-missing-values/>
- [4] GeeksforGeeks. *Understanding Logistic Regression*. <https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>
- [5] GeeksforGeeks. *Principal Component Analysis (PCA)*. <https://www.geeksforgeeks.org/machine-learning/principal-component-analysis-pca/>
- [6] GeeksforGeeks. *AUC-ROC Curve*. <https://www.geeksforgeeks.org/machine-learning/auc-roc-curve/>